# LLM×MapReduce: Simplified Long-Sequence Processing using Large Language Models

**Zihan Zhou**[1*], **Chong Li**[2*], **Xinyi Chen**[3*], **Shuo Wang**[4†], **Yu Chao**[4], **Zhili Li**[5], **Haoyu Wang**[5], **Rongqiao An**[4], **Qi Shi**[4], **Zhixing Tan**[4], **Xu Han**[4,6,7], **Xiaodong Shi**[1†], **Zhiyuan Liu**[4,6,7†], **Maosong Sun**[4,6,7]
[1]Xiamen University [2]Peking University [3]Nankai University
[4]Dept. of Comp. Sci. & Tech., Tsinghua University [5]BUPT
[6]Institute for AI, Tsinghua University
[7]Beijing National Research Center for Information Science and Technology

arXiv:2410.09342v1 [cs.CL] 12 Oct 2024

## Abstract

Enlarging the context window of large language models (LLMs) has become a crucial research area, particularly for applications involving extremely long texts. In this work, we propose a novel training-free framework for processing long texts, utilizing a divide-and-conquer strategy to achieve comprehensive document understanding. The proposed LLM×MapReduce framework splits the entire document into several chunks for LLMs to read and then aggregates the intermediate answers to produce the final output. The main challenge for divide-and-conquer long text processing frameworks lies in the risk of losing essential long-range information when splitting the document, which can lead the model to produce incomplete or incorrect answers based on the segmented texts. Disrupted long-range information can be classified into two categories: inter-chunk dependency and inter-chunk conflict. We design a *structured information protocol* to better cope with inter-chunk dependency and an *in-context confidence calibration* mechanism to resolve inter-chunk conflicts. Experimental results demonstrate that LLM×MapReduce can outperform representative open-source and commercial long-context LLMs, and is applicable to several different models.[1]

## 1 Introduction

Large language models (LLMs) exhibit impressive performance across a wide range of complex tasks (OpenAI, 2023), including question answering (Anthropic, 2023), code generation (Luo et al., 2024), and solving mathematical problems (Luo et al., 2023). However, due to their quadratic computational complexity and a lack of high-quality long training examples, most LLMs are trained with a limited window size (Touvron et al., 2023a,b; Jiang et al., 2023). This context limit restricts the application of modern LLMs to long-sequence processing tasks. In response to this issue, several researchers have focused on extending the context length of LLMs. Existing studies can be broadly categorized into two types: training-based and training-free methods.

For training-based extension methods, it is necessary to prepare long training data and allocate substantial computational resources to support the additional training. Xiong et al. (2023) propose adjusting the base frequency of RoPE (Su et al., 2023), and then training the model with 400 billion tokens of long text, significantly improving its performance on long-context tasks. Chen et al. (2024) introduce LongLoRA, which employs shifted sparse attention for efficient fine-tuning and utilizes learnable embedding and normalization layers during long-context fine-tuning. Although these training-based methods can effectively extend the context length of LLMs, they may be inapplicable in scenarios where sufficient computational resources and high-quality long texts are unavailable.

By contrast, training-free context extension approaches aim to break the length limit of LLMs without tuning their parameters. For example, Xiao et al. (2024b) suggest preserving several initial tokens within a sliding window attention mechanism, thereby enabling large language models to process unlimited text without the need for fine-tuning. InfLLM (Xiao et al., 2024a), also leveraging sliding window attention, employs additional memory units to store distant contexts and incorporates an efficient mechanism for retrieving relevant historical information. Another prominent research direction employs the divide-and-conquer idea, processing long sequences by splitting them into shorter chunks. LangChain (Chase, 2022) initially introduces the MapReduce method, where text segments are processed in parallel during the

[*]Equal Contribution.
[†]Correspondence to Shuo Wang, Xiaodong Shi, and Zhiyuan Liu.
[1]The code is available at https://github.com/thunlp/LLMxMapReduce.

map stage, followed by the aggregation of intermediate results across all segments to predict the final output. Similarly, in XL$^3$M (Wang et al., 2024), long texts are divided into multiple short sub-contexts, each paired with a question. Relevant segments are then selected using LLMs and combined chronologically to generate the final answer. The major challenge for this kind of method is that different segments are processed independently, which may break some essential long-range information. Disrupted long-range information can be divided into two categories: (1) *inter-chunk dependency*, where evidence is spread across different chunks and relies on each other; and (2) *inter-chunk conflict*, where evidence across chunks is contradictory, requiring the model to resolve these conflicts in order to predict the final answer.

To address the challenges of inter-chunk dependency and inter-chunk conflict, recent works have proposed more advanced divide-and-conquer frameworks. LongAgent (Zhao et al., 2024) introduces a framework comprising a leader agent and multiple member agents, each responsible for processing a chunk, all powered by LLMs. Each member provides an answer to the leader, who groups the responses and randomly selects a representative from each group to determine the final answer. However, our experiments show that LongAgent's aggregation mechanism does not effectively resolve inter-chunk dependency and conflict, as randomly selecting members can result in the loss of important evidence. Unlike LongAgent, which processes multiple chunks in parallel, Chain-of-Agents (CoA) (Zhang et al., 2024b) sequentially processes split chunks using an accumulated summary. However, because CoA's workflow does not explicitly address the inter-chunk conflict problem, it performs worse than LongAgent in our experiments. LC-Boost (Qian et al., 2024) defines an action space and selects appropriate actions for sequentially processing chunks. To address inter-chunk conflicts, LC-Boost adaptively either appends new evidence or updates the summary. However, in complex cases where historical and current information conflict, LC-Boost may struggle to fully resolve the issue relying solely on the accumulated summary and the current text.

In this paper, we introduce LLM×MapReduce, a training-free framework for processing long texts that utilizes a divide-and-conquer approach, allowing models with short context windows to effectively handle long contexts. To address the challenges of inter-chunk dependency and conflict, we introduce a *structured information protocol* and an *in-context confidence calibration* mechanism. The structured information protocol defines the information passed from the map stage to the reduce stage, ensuring the model has the critical inputs needed to infer the correct answer when aggregating different chunks. In-context confidence calibration allows the model to assign a reliable confidence score to the output of each chunk, aiding in effectively resolving inter-chunk conflicts. We evaluate the proposed method on various long-text benchmarks, and the experimental results show that our approach outperforms both closed- and open-source LLMs in terms of both performance and efficiency. Through ablation experiments, we further validate the effectiveness of each component in LLM×MapReduce, demonstrating how each piece contributes to the overall performance.

## 2 Approach

### 2.1 Problem Description

In real-world scenarios, users may require the LLM to comprehend one or more lengthy documents that far exceed the model's effective context window. Formally, let $X$ represent the user-provided long text and $L$ denote the model's effective context length. In this work, we focus on cases where $|X| \gg L$, where $|X|$ represents the length of $X$. we partition the input text $X$ into a series of chunks $\{x_1, x_2, \cdots, x_n\}$, where the length of each chunk $x_i$ is within the model's effective context length $L$. For a given user query $Q$, the LLM, parameterized by $\boldsymbol{\theta}$, processes each chunk to generate intermediate outputs, which are then aggregated to predict the final answer.

### 2.2 Workflow of LLM × MapReduce

Figure 1 depicts the overall framework of the proposed LLM×MapReduce framework. Like LangChain (Chase, 2022), the LLM×MapReduce workflow consists of three stages: map, collapse, and reduce. During the map stage, we utilize an LLM as the map model to extract the necessary information for each chunk $x_i$:

$$s_i = f_{\text{map}}\left(x_i, Q; \boldsymbol{\theta}\right), \tag{1}$$

where $Q$ is the user query and $f_{\text{map}}$ represents the map function powered by the LLM, parameterized by $\boldsymbol{\theta}$. Our experiments show that the design of

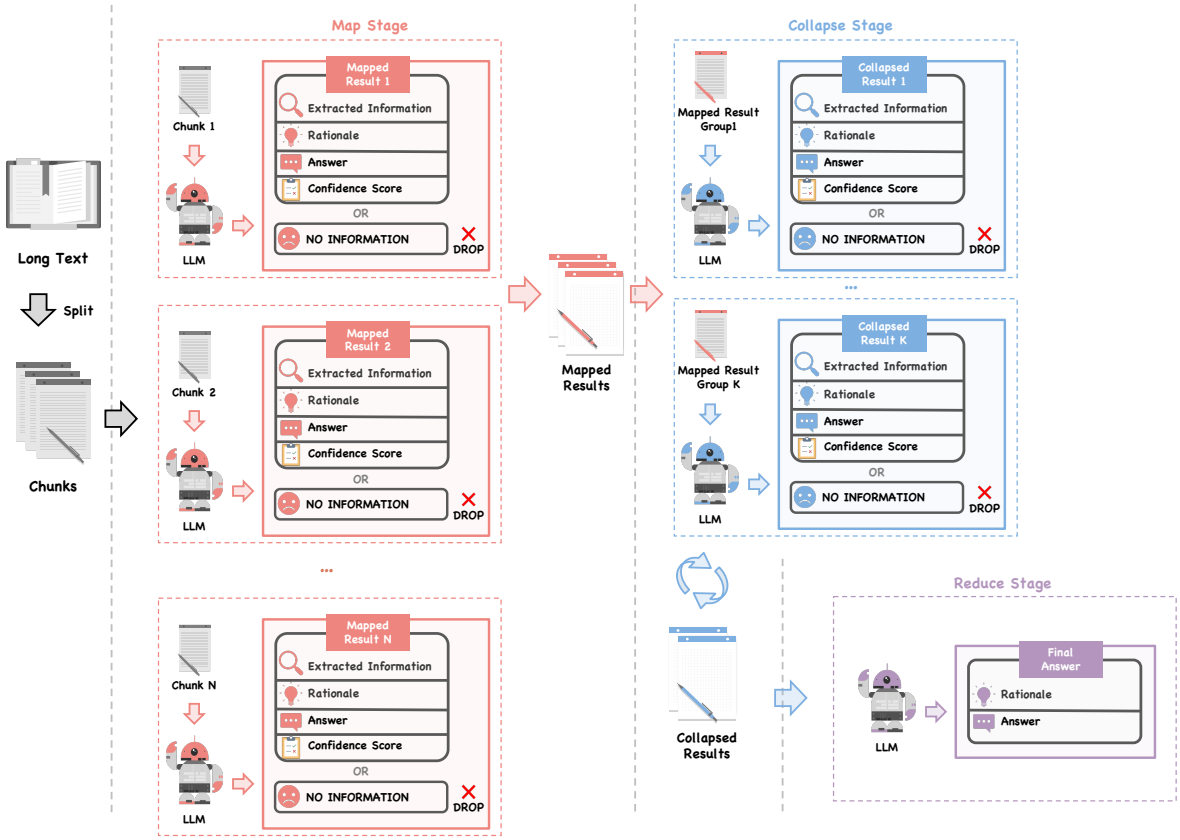


Figure 1: Overview of the proposed LLM×MapReduce framework. After dividing the provided long text into a series of chunks, the model processes each chunk to extract an information structure containing the essential content needed to address the query. This is referred to as the map stage in our framework. The mapped results are then compressed during the collapse stage, preparing them for the reduce stage. The structure of the collapsed results mirrors that of the mapped results. The collapse stage ensures that the input to the reducing model remains within its effective length (i.e., $L$). Based on the structured outputs from the first two stages (i.e., the map and collapse stages), the reduce model aggregates information from all chunks, resolves inter-chunk conflicts using calibrated confidence scores, and predicts the final answer.

the mapped results, $\{s_1, \cdots, s_N\}$, is crucial for enabling the divide-and-conquer framework to effectively comprehend long documents. In this work, we propose a structured information protocol aimed at improving communication efficiency between the different stages.

In some cases, the input text is extremely long, resulting in mapped results that still exceed the context window of the LLM being used. To address this, a collapse stage is employed to compress the mapped results. We divide the $N$ mapped results into $K$ groups, ensuring that the length of each group remains within the model's context window $L$. For the $j$-th group of mapped results $g_j$, we leverage an LLM to output a compact result:

$$c_j = f_{\text{collapse}}\left(g_j, Q; \boldsymbol{\theta}\right). \tag{2}$$

It is important to note that the structure of each collapsed result $c_j$ remains the same as that of each mapped result $s_i$. If the total length of the mapped results $\{s_1, \cdots, s_N\}$ is less than $L$, we use the mapped results directly as the collapsed results for the reduce stage. If the collapsed results $\{c_1, \cdots, c_K\}$ still exceed $L$, we iteratively apply the collapse function $f_{\text{collapse}}$ until their length is reduced to less than $L$. Briefly, we use $\{c_1, \cdots, c_K\}$ to denote the final output of the collapse stage.

Finally, in the reduce stage, the final response is generated based on the collapsed results:

$$a = f_{\text{reduce}}\left(\{c_1, \cdots, c_K\}, Q; \boldsymbol{\theta}\right). \tag{3}$$

In LLM×MapReduce, we do not need to tune the model parameters $\boldsymbol{\theta}$. Instead, the three functions (i.e., $f_{\text{map}}$, $f_{\text{collapse}}$, and $f_{\text{reduce}}$) are implemented using prompts with existing LLMs.

The aforementioned divide-and-conquer framework is quite straightforward for long text processing, and has been explored in some previous stud-

ies (Chase, 2022; Zhao et al., 2024; Zhang et al., 2024b). However, in our experiments, we find that simply combining an LLM and the divide-and-conquer strategy can not achieve satisfying performance on modern long-text benchmarks (Zhang et al., 2024a; Hsieh et al., 2024).

The major challenge is that segmenting the entire document may disrupt crucial long-range clues. The disrupted long-range information can be divided into two categories: inter-chunk dependency and inter-chunk conflicts. We therefore focus on enhancing the divide-and-conquer framework's ability to process cross-chunk information. Specifically, we propose a structured information protocol to address inter-chunk dependencies and an in-context confidence calibration mechanism to resolve inter-chunk conflicts. These approaches will be explained in the following subsections.

### 2.3 Structured Information Protocol

An important research question for divide-and-conquer long-text processing frameworks is determining *what information the map stage should convey to the reduce stage*. If the mapped results are overly simplified, as seen in LongAgent (Zhao et al., 2024), they may miss crucial details needed for subsequent stages (e.g., the reduce stage) to effectively handle inter-chunk dependencies and conflicts. On the other hand, if the mapped results are too complex, they introduce significant computational overhead, increasing the overall latency of the framework. Additionally, excessive unrelated information may interfere with the reduce model's ability to produce the correct answer.

To this end, we introduce a specialized information structure consisting of four components:

- **Extracted Information**: key facts or data relevant to the query $Q$ that are extracted from the current chunk, providing the necessary background for subsequent stages to address inter-chunk dependencies.
- **Rationale**: the analysis or inference process that explains how the model derives the intermediate answer from the extracted information, helping to mitigate the risk of hallucinations in subsequent stages.
- **Answer**: the intermediate answer to the query, derived from the extracted information and rationale. If, after providing the rationale, the model determines that the passage does not contain relevant information to address the question, it will output "`NO INFORMATION`", which will be disregarded in subsequent stages.
- **Confidence Score**: a score (out of 5) reflecting the model's confidence in the answer, indicating the completeness and reliability of the information. The confidence score is important for resolving inter-chunk conflicts.

To maintain a consistent input format for the reduce stage, both the map and collapse stages produce data in the structured format described above. "Extracted Information" together with "Rationale" provide the essential supplementary details needed for the reduce model to integrate answers from different chunks. Ablation experiments in Section 3.3 demonstrate the effectiveness of the structured information protocol in enhancing the model's ability to manage inter-chunk dependencies.

A remaining issue with the structured information protocol is the potential inconsistency in confidence scores estimated across different chunks when resolving inter-chunk conflicts. Without a general criterion for confidence estimation, the model may assign varying levels of confidence to different chunks, even if the content is equally reliable. We thus propose an in-context confidence calibration mechanism to align the confidence scores of different chunks to a consistent standard.

### 2.4 In-Context Confidence Calibration

When long sequences are split into multiple chunks, conflicts may arise due to incomplete information. The aforementioned information structure provides a confidence score for the answer from each chunk. During the collapse and reduce stages, confidence scores are crucial for guiding the merging of information and generating both the rationale and the final answer.

To make the confidence scores across different chunks comparable, we propose to calibrate them through in-context learning, without adjusting model parameters. Specifically, we provide confidence estimation principles alongside a typical example for different levels of confidence score. By referring to the principles and the examples, the model is expected to apply a consistent criterion when processing different chunks. Figure 2 provides an example of the calibration prompt. We can customize different calibration prompts for various tasks. Claims fully supported by the provided text

are assigned high confidence, while those inferred by the model receive medium confidence. Claims not related to the provided text are assigned low confidence. Experiments in Section 3.3 demonstrate the necessity of the proposed in-context confidence calibration mechanism.

```
Assign a confidence score (out of 5) to
your answer based on the completeness and
reliability of the extracted information
and your rationale. The following is some
assigning scoring cases:
<Text: [ Jerry is 18 years old this year. He can
swim and wants to be an athlete. ].
Examples of confidence estimation: [
Jerry can swim, 5 points;
Jerry will become an athlete in the future, 3.5
points;
Jerry will become a swimming athlete in the fu-
ture, 3 points;
Jerry is strong, 3 points;
Jerry can play chess, 0 points;
Jerry likes talking, 0 points ] >.
```

Figure 2: Prompt for in-context confidence calibration.

## 3 Experiments

### 3.1 Setup

**Models** We use two well-known open-source models to validate the effectiveness of the proposed LLM×MapReduce framework, which are Llama3-70B-Instruct[2] and Qwen2-72B-Instruct[3]. We employ vLLM[4] for model inference, and the decoding temperature is set to 0.7.

**Evaluation** We evaluate the performance of the involved models and methods on InfiniteBench (Zhang et al., 2024a), where the average input length exceeds 100K tokens. This benchmark assesses the long-text capabilities of LLMs across several dimensions, including long-range retrieval, language comprehension, code understanding, and mathematical problem-solving. We exclude the subsets `Code.Run` and `Math.Calc`, as nearly all models achieve less than 5% accuracy on these tasks, making it difficult to differentiate performance among the models. We utilize the evaluation code open-sourced by Zhang et al. (2024a) to calculate scores, with the exception of `En.Dia`. We find that the recall score for this task tends to increase with longer model outputs. Therefore, we directly engage two human experts with experience in natural language processing to manually assess the accuracy. For `Retrieve.PassKey`, `Retrieve.Number`, `Retrieve.KV`, we use the retrieval prompt. For `En.Sum`, we use the summarization prompt. For `En.QA`, `En.MC`, `En.Dia`, we use the language question-answering prompt. For `Code.Debug`, we use the code prompt. For `Math.Find`, we use the math prompt.

**Baselines** We select several representative models and methods as our baselines. For closed-source models, we compare against GPT-4, Claude 2 (Anthropic, 2023), and Kimi-Chat. For open-source models, we include YaRN-Mistral[5], Yi-6B-200K, Yi-34B-200K[6], and Qwen2-72B-Instruct. Additionally, we compare LLM×MapReduce with two recent representative frameworks for divide-and-conquer long-sequence processing: LongAgent (Zhao et al., 2024) and Chain-of-Agents (Zhang et al., 2024b).

### 3.2 Main Results

Table 1 presents the performance of the involved methods on InfiniteBench. Among closed-source models, GPT-4 achieves the highest average score of 57.34. In the open-source category, Qwen2-72B-Instruct, which has a claimed effective length of 128K, achieves an average score of 54.74, even surpassing Claude 2 and Kimi-Chat.

For the divide-and-conquer methods, the backbone model used is Llama3-70B-Instruct, which has an effective context length of 8K, significantly shorter than the test examples in InfiniteBench. The results indicate that LongAgent (Zhao et al., 2024) outperforms CoA on nearly all subtasks. The average score of LongAgent is close to that of Qwen2-72B-Instruct (53.81 vs. 54.74). Surprisingly, the proposed LLM×MapReduce method achieves the highest average score (68.66), outperforming both the closed-source models and the divide-and-conquer baselines. Augmented by the proposed method, Llama3-70B-Instruct performs well on all the subtasks. Additionally, our method is compatible with Qwen2-72B-Instruct, demonstrating its generalization capability.

[2] https://huggingface.co/meta-llama/Meta-Llama-3-70B-Instruct

[3] https://huggingface.co/Qwen/Qwen2-72B-Instruct

[4] https://github.com/vllm-project/vllm

[5] https://huggingface.co/NousResearch/Yarn-Mistral-7b-128k

[6] https://huggingface.co/01-ai

| Methods | Re.Pa | Re.Nu | Re.KV | En.Sum | En.QA | En.MC | En.Dia | Co.De | Ma.Fi | Avg. |
|---|---|---|---|---|---|---|---|---|---|---|
| *Closed-Source Models* | | | | | | | | | | |
| GPT-4$^\star$ | **100.00** | **100.00** | 89.00 | 14.73 | 22.44 | 68.12 | 7.50 | 54.31 | 60.00 | 57.34 |
| Claude 2$^\star$ | 97.80 | 99.15 | 65.40 | 14.50 | 11.97 | 67.25 | 43.00 | 33.24 | 32.29 | 51.62 |
| Kimi-Chat | 99.32 | 97.45 | 69.20 | 29.94 | 18.81 | 79.91 | 15.50 | 38.32 | 18.57 | 51.89 |
| *Open-Source Models* | | | | | | | | | | |
| YaRN-Mistral$^\star$ | 92.71 | 58.31 | 0.00 | 9.09 | 9.55 | 29.26 | 4.50 | 23.60 | 17.14 | 27.13 |
| Yi-6B-200K$^\star$ | **100.00** | 94.92 | 0.00 | 0.92 | 9.20 | 36.68 | 1.50 | 18.78 | 4.29 | 29.59 |
| Yi-34B-200K$^\star$ | **100.00** | **100.00** | 0.00 | 1.33 | 12.17 | 46.29 | 3.50 | 21.32 | 25.71 | 34.48 |
| Q2-72B-I | **100.00** | **100.00** | 29.00 | 31.85 | 21.97 | 81.66 | 23.00 | 45.43 | 59.71 | 54.74 |
| *Divide-and-Conquer Frameworks* | | | | | | | | | | |
| L3-70B-I+LA | 99.32 | 93.05 | 74.60 | 2.19 | **35.41** | 69.00 | 7.50 | 24.11 | 79.14 | 53.81 |
| L3-70B-I+CoA | 9.32 | 15.59 | 1.80 | 10.10 | 7.03 | 27.51 | 9.50 | 18.27 | 44.57 | 15.97 |
| L3-70B-I×MR | **100.00** | 99.79 | **98.89** | 30.63 | 34.70 | 82.10 | 17.50 | **62.94** | **91.43** | **68.66** |
| Q2-72B-I×MR | **100.00** | **100.00** | 98.80 | **32.39** | 23.13 | **83.84** | **46.50** | 54.82 | 54.29 | 65.97 |

Table 1: Results on InfiniteBench. "$^\star$" indicates that we directly use the model outputs released by Zhang et al. (2024a) and re-calculate the score. "Q2-72B-I" and "L3-70B-I" refer to Qwen2-72B-Instruct and Llama3-70B-Instruct, respectively. "LA" and "CoA" denote LongAgent (Zhao et al., 2024) and Chain-of-Agents (Zhang et al., 2024b), which are two recent representative frameworks for divide-and-conquer long-sequence processing .

### 3.3 Ablation Study

As mentioned in Section 1, the major challenge for divide-and-conquer long-sequence processing methods lies in addressing inter-chunk dependencies and resolving inter-chunk conflicts. In LLM×MapReduce, we introduce a structured information protocol and an in-context confidence calibration mechanism, setting our method apart from existing divide-and-conquer baselines. We conduct ablation experiments to investigate the effect of the two components.

As shown in Table 2, removing the in-context confidence calibration mechanism leads to a performance decline across all tasks, particularly in English language understanding tasks (i.e., En.Avg). When both confidence calibration and the structured information protocol are disabled, the performance drops even more significantly compared to the full framework. For Re.Avg, there is a slight decrease from 99.56 to 97.14, but En.Avg experiences a much larger drop, falling by 15.30 points to 25.93. In Co.De, performance decreases substantially by 16.49 points, and in Ma.Fi, the score drops sharply by 35.43 points. These results underscore the importance of both mechanisms in maintaining strong performance for long-sequence processing.

| Method | Re.Avg | En.Avg | Co.De | Ma.Fi |
|---|---|---|---|---|
| L3-70B-I×MR | 99.56 | 41.23 | 62.94 | 91.43 |
| -Conf. | 96.00 | 39.18 | 58.12 | 90.00 |
| -Struc. | 97.14 | 25.93 | 46.45 | 56.00 |

Table 2: Effect of structured information protocol and in-context confidence calibration. "Re.Avg" and "En.Avg" denote the average performance on retrieval tasks and English language understanding tasks, respectively.

### 3.4 Extremely Long Evaluation

Needle-in-a-haystack (NIAH) (Kamradt, 2023) is a widely-used method for evaluating the ability of LLMs to handle long texts by identifying specific facts within long documents. To assess the performance of our framework in handling extremely long texts, we extend the NIAH test to a length of 1280K million tokens.

Figure 3 presents the results, showing that our proposed method enables Llama3-70B-Instruct, which has a trained context length of 8K tokens, to effectively deal with sequences up to 1280K tokens. This demonstrates the potential of our framework for processing extremely long sequences.

### 3.5 Inference Latency

Since divide-and-conquer long-sequence processing frameworks introduce multiple intermediate steps, they may be slower than standard decoding. To assess this, we measure the inference latency of the different approaches using 20 test examples, each with 128K tokens. Since the original Llama3-70B-Instruct does not support 128K tokens, we

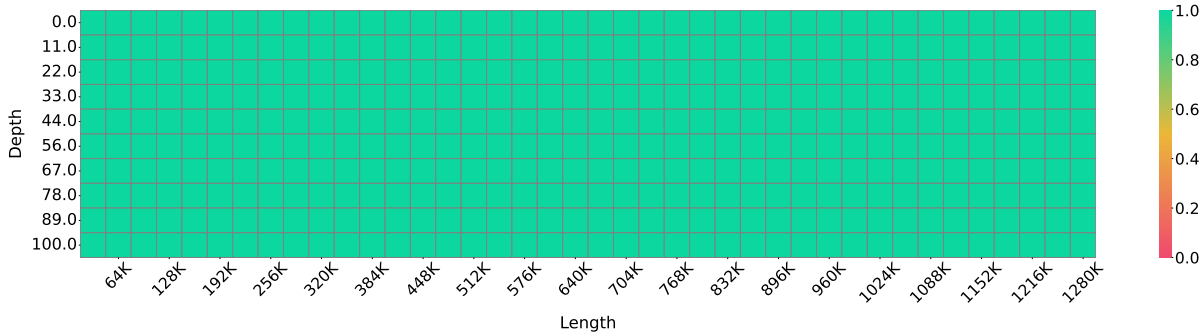


Figure 3: Performance of Llama3-70B-Instruct×MapReduce on the NIAH test, with the maximum length of the haystack set to 1280K tokens.

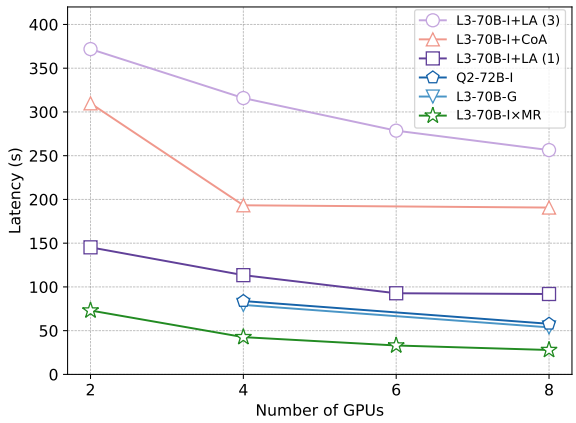


Figure 4: Comparison of inference latency. "L3-70B-G" represents Llama3-70B-Instruct-Gradient-1048K.

use Llama3-70B-Instruct-Gradient-1048K[7], an extended version of Llama3-70B-Instruct, to evaluate the inference speed. Since LongAgent uses a multi-turn mechanism to resolve inter-chunk conflicts, we report the latency for LongAgent with the maximum number of turns set to 1 and 3. The experiments are conducted using NVIDIA A100 GPUs (80 GB).

As shown in Figure 4, both CoA and LongAgent are slower than standard decoding across different settings. However, a notable advantage of divide-and-conquer methods is their lower GPU requirements for handling long sequences. For standard decoding, at least 4 GPUs are needed to process 128K tokens, whereas divide-and-conquer methods can support 128K tokens using just 2 GPUs. Surprisingly, the proposed LLM×MapReduce framework outperforms not only other divide-and-conquer baselines in speed but also standard decoding. The efficiency of our method is achieved by avoiding the need to repeatedly process text chunks to resolve conflicts, as seen in LongAgent. Instead, we employ a structured information protocol and an in-context confidence calibration mechanism to effectively integrate information across chunks.

## 4 Conclusion

In this work, we introduce an effective divide-and-conquer framework for long-sequence processing, LLM×MapReduce. With the structured information protocol and the in-context confidence calibration mechanism, LLM×MapReduce effectively handles long texts, surpassing standard long-context LLMs and other divide-and-conquer baselines in terms of both effectiveness and efficiency.

[7] https://huggingface.co/gradientai/Llama-3-70B-Instruct-Gradient-1048k

## References


Anthropic. 2023. Model card and evaluations for claude models.

Harrison Chase. 2022. Langchain. https://github.com/langchain-ai/langchain.

Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2024. Longlora: Efficient fine-tuning of long-context large language models. *Preprint*, arXiv:2309.12307.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. 2024. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling*.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b. *Preprint*, arXiv:2310.06825.

Greg Kamradt. 2023. Llms need needle in a haystack test-pressure testing llms.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *Preprint*, arXiv:2308.09583.

Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2024. Wizardcoder: Empowering code large language models with evol-instruct. In *The Twelfth International Conference on Learning Representations*.

OpenAI. 2023. Gpt-4 technical report. *ArXiv*, abs/2303.08774.

Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Yujia Zhou, Xu Chen, and Zhicheng Dou. 2024. Are long-llms a necessity for long-context tasks? *Preprint*, arXiv:2405.15318.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2023. Roformer: Enhanced transformer with rotary position embedding. *Preprint*, arXiv:2104.09864.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023a. Llama: Open and efficient foundation language models. *Preprint*, arXiv:2302.13971.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023b. Llama 2: Open foundation and fine-tuned chat models. *Preprint*, arXiv:2307.09288.

Shengnan Wang, Youhui Bai, Lin Zhang, Pingyi Zhou, Shixiong Zhao, Gong Zhang, Sen Wang, Renhai Chen, Hua Xu, and Hongwei Sun. 2024. Xl3m: A training-free framework for llm length extension based on segment-wise inference. *Preprint*, arXiv:2405.17755.

Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. 2024a. Infllm: Training-free long-context extrapolation for llms with an efficient context memory. *Preprint*, arXiv:2402.04617.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024b. Efficient streaming language models with attention sinks. *Preprint*, arXiv:2309.17453.

Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, Madian Khabsa, Han Fang, Yashar Mehdad, Sharan Narang, Kshitiz Malik, Angela Fan, Shruti Bhosale, Sergey Edunov, Mike Lewis, Sinong Wang, and Hao Ma. 2023. Effective long-context scaling of foundation models. *Preprint*, arXiv:2309.16039.

Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024a. $\infty$bench: Extending long context evaluation beyond 100k tokens. *Preprint*, arXiv:2402.13718.

Yusen Zhang, Ruoxi Sun, Yanfei Chen, Tomas Pfister, Rui Zhang, and Sercan Ö. Arik. 2024b. Chain of agents: Large language models collaborating on long-context tasks. *Preprint*, arXiv:2406.02818.

Jun Zhao, Can Zu, Hao Xu, Yi Lu, Wei He, Yiwen Ding, Tao Gui, Qi Zhang, and Xuanjing Huang. 2024. Longagent: Scaling language models to 128k context through multi-agent collaboration. *Preprint*, arXiv:2402.11550.